

Team Member Names: Hank Atherton, Weronika Golden

Architecture explanation and architecture diagram

Overview of the Chosen Architecture

The SunDevil Connect system is designed using a Client–Server architecture combined with a layered structure based on the Boundary–Control–Entity (BCE) model. This architectural choice enables a clear separation between user interaction, application logic, and data management, while also supporting distributed communication between a client application and a backend server.

At a high level, the system consists of a JavaFX-based client application, a Java backend server, a SQLite database for persistent storage, and gRPC, which uses Protocol Buffers for communication between the client and server. This design supports the system’s primary goal of providing a centralized platform where students can discover events, join clubs, and engage with campus activities efficiently.

In addition, the system is organized into three distinct modules: client, server, and shared. The client module contains all JavaFX user interface components, the server module manages business logic and data processing, and the shared module contains Protocol Buffer (gRPC) definitions used to define the structure of messages exchanged between the client and server. This modular separation enforces strict system boundaries at compile time, improving maintainability and reducing coupling between components.

By combining Client–Server with BCE layering, the system benefits from both physical separation (client vs. server) and logical separation (UI vs. logic vs. data), resulting in a highly modular and maintainable architecture.

Layered Architecture Using the BCE Model

The system is organized into three logical layers, each with a clearly defined responsibility. This separation ensures that changes in one part of the system do not unnecessarily affect others.

Boundary Layer (Presentation Layer)

The Boundary layer contains all user interface components responsible for interacting with users and displaying system data. These components are implemented in the JavaFX client application and include:

- LoginPage
- ClubDashboard
- ClubPageView
- ClubBrowseView
- EventListView

- EventFilterPanel
- EventDetailsView
- AnnouncementListView
- AdminPanel
- NotificationView

These components:

- Handle user input and display information
- Communicate with controllers via structured requests
- Do not contain business logic

This design ensures that UI changes (such as layout updates or new views) can be made independently of the system's core logic.

Control Layer (Application Logic Layer)

The Control layer acts as the “brain” of the system and is responsible for coordinating workflows and enforcing business rules. Key controllers include:

- AuthenticationController
- ClubMembershipController
- ClubBrowsingController
- EventBrowsingController
- EventRegistrationController
- EventManagementController
- AnnouncementController
- ClubApproveController
- ContentModerationController

These controllers:

- Receive requests from the Boundary layer
- Process business logic and enforce rules (e.g., event capacity, membership approval)
- Interact with the Entity layer and backend services

This layer ensures that system behavior remains consistent and centralized, making it easier to debug, test, and extend.

Entity Layer (Data Layer)

The Entity layer represents the core data structures of the system. These objects correspond directly to persistent data stored in the database. Examples include:

- User (Student, Admin)
- Club
- ClubMembership
- MembershipRequest

- Event
- EventRegistration
- Announcement
- Content
- Notification

These entities:

- Represent real-world concepts within the system
- Store and manage system data
- Are accessed and modified through the Control layer

This structure aligns with the object classification defined earlier in the project and ensures a strong foundation for data consistency .

Client–Server Communication

A key component of this architecture is the separation between the client and server. The JavaFX client is responsible for presentation, while the backend server handles logic and data processing.

Communication between the client and server is implemented using gRPC, which leverages Protocol Buffers for efficient data serialization. This allows the client to directly invoke server-side methods through remote procedure calls (RPCs), eliminating the need to manually manage networking, sockets, or message routing.

gRPC also supports streaming communication, which is particularly useful for real-time features such as notifications. In this system, notifications are constructed on the server and streamed to the client, where they are rendered in the user interface. This aligns with the Observer design pattern and enables responsive, real-time updates.

This choice improves performance and prepares the system for future scalability, such as supporting mobile or web clients.

Benefits of the Chosen Architecture

The combination of Client–Server and BCE layering provides several important advantages:

1. Clear Separation of Concerns

- UI, logic, and data are fully decoupled
- Changes in one layer do not affect others
- Improves readability and maintainability

2. Scalability

- Client and server can be developed and scaled independently
- New clients (e.g., mobile apps) can be added without modifying backend logic

- Supports future system expansion

3. Performance Optimization

- Protocol Buffers enable efficient data serialization, while gRPC improves performance by handling networking, threading, and communication overhead automatically
- SQLite provides lightweight and fast data storage
- Backend processing reduces client workload

4. Modularity and Extensibility

- Design patterns such as Strategy, Observer, Singleton, and Factory integrate seamlessly into this architecture
- New features can be added with minimal changes to existing components
- Encourages reusable and maintainable code

5. Testability

- Controllers can be tested independently of the UI
- Backend services can be tested without running the client
- Improves reliability and debugging efficiency

Comparison with Alternative Architecture

An alternative approach to designing the SunDevil Connect system would be a monolithic architecture, in which all components—including the user interface, business logic, and data access—are tightly integrated into a single application. While this approach may be simpler to implement initially, it introduces significant limitations as the system grows in complexity. In a monolithic design, components are highly coupled, meaning that changes in one part of the system can unintentionally impact other parts. This tight coupling makes the system more difficult to maintain over time, especially as new features are added or existing functionality is modified. Additionally, scalability is limited because the system cannot easily separate client and server responsibilities, making it more challenging to distribute workloads or support multiple types of clients. Testing also becomes more complex in a monolithic system, as individual components cannot be easily isolated, requiring larger portions of the system to be tested together.

In contrast, the chosen Client–Server architecture combined with the BCE layered model provides a much more flexible and maintainable solution. By separating the system into distinct layers and distributing responsibilities between the client and server, the architecture promotes strong modularity, allowing components to be developed, updated, and managed independently. This separation also enhances scalability, as the client and server can be deployed and scaled independently based on system demand. Furthermore, the architecture supports greater flexibility by allowing new technologies or platforms, such as mobile or web clients, to be integrated without requiring significant changes to the backend. Performance is also improved through the use of gRPC with Protocol Buffers, which reduces communication overhead and enables efficient client–server interaction. Overall, the clear structure of the Client–Server and BCE

approach reduces system complexity, simplifies maintenance, and provides a strong foundation for future growth. For these reasons, it is significantly better suited for a dynamic and evolving system like SunDevil Connect.

Conclusion

The SunDevil Connect system leverages a Client–Server architecture combined with the BCE layered model to achieve a clean, modular, and scalable design. This architecture effectively separates concerns, improves system performance, and supports future extensibility.

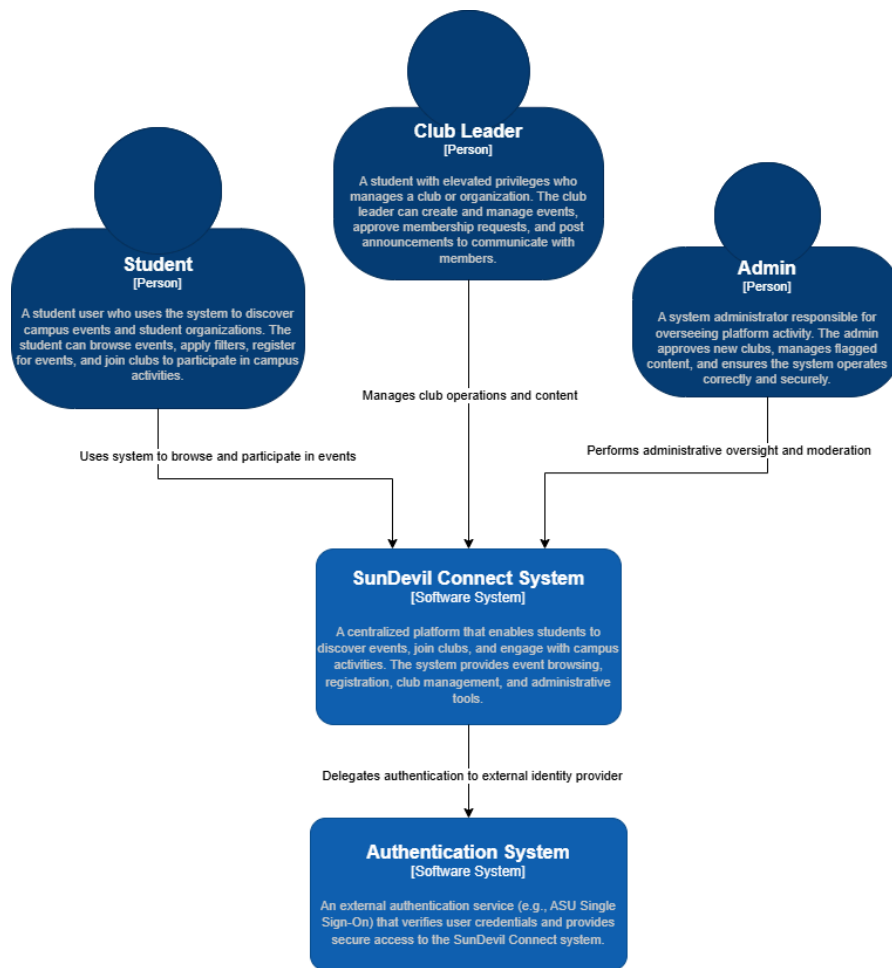
By using gRPC with Protocol Buffers for communication, SQLite for data storage, and well-defined design patterns within the layered structure, the system is well-positioned to handle the complexity of managing student organizations and events. Compared to a monolithic approach, this architecture provides clear advantages in maintainability, scalability, and flexibility, making it the optimal choice for this project.

Architecture Diagrams

Context Diagram

The context diagram illustrates the high-level interaction between external users and the SunDevil Connect system. The primary actors include students, club leaders, and administrators, each interacting with the system in different ways based on their roles. Students use the platform to browse events, join clubs, and participate in campus activities. Club leaders manage organizational operations, including creating events, approving memberships, and posting announcements. Administrators oversee system activity, moderate content, and approve new clubs.

At the center of the diagram, the SunDevil Connect system acts as the core platform that facilitates all interactions. It provides functionality for event discovery, registration, and club management while maintaining a centralized and user-friendly experience. The system also integrates with an external authentication system, which is responsible for verifying user credentials and ensuring secure access. This diagram emphasizes the system boundary and clearly defines how external entities interact with the platform without exposing internal implementation details.



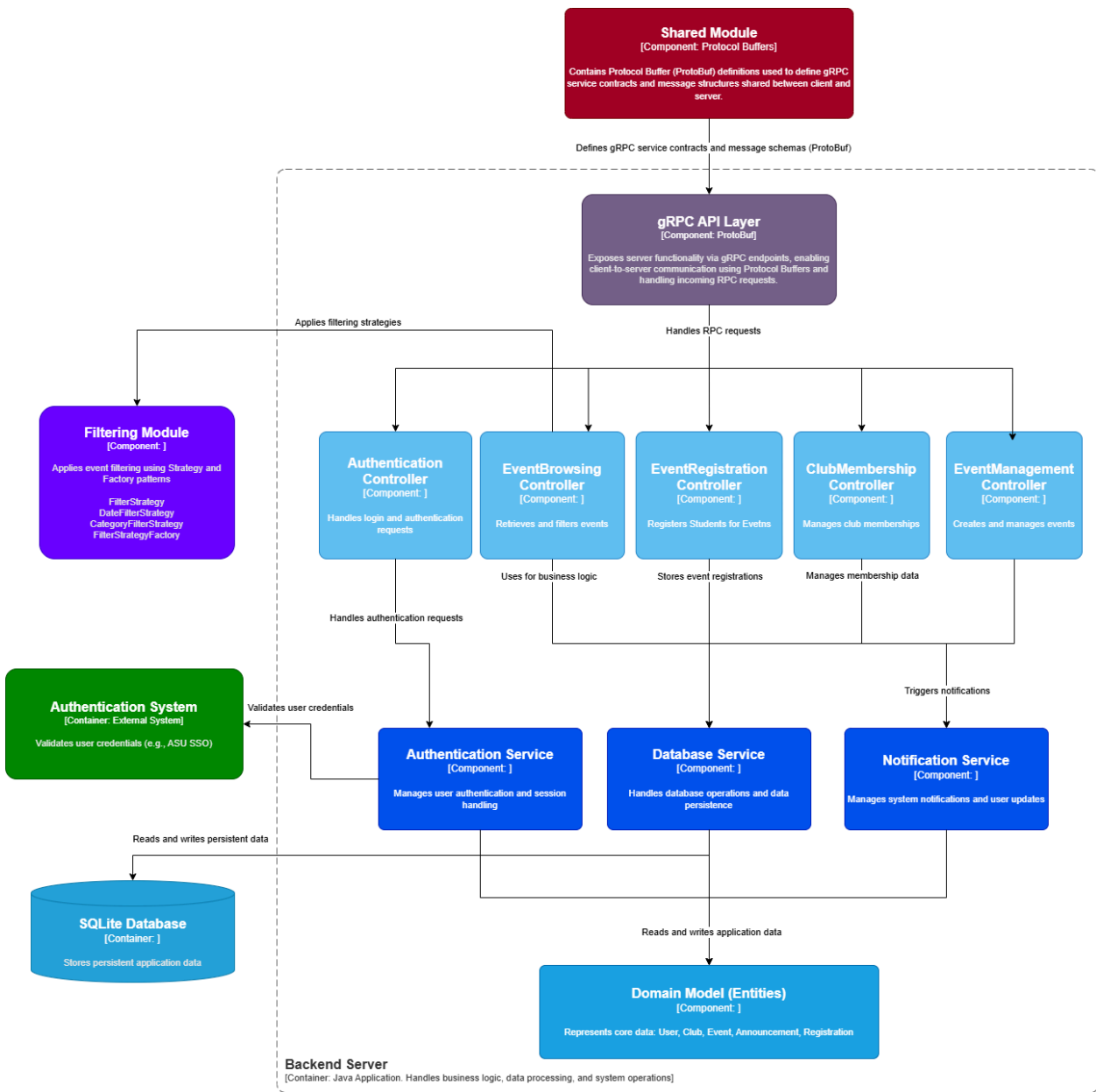
Component Diagram

The component diagram provides a detailed view of the internal structure of the SunDevil Connect backend system, following the C4 Level 3 model. The system is organized into logical components that align with the Boundary–Control–Entity (BCE) architecture. At the top layer, controller components such as AuthenticationController, EventBrowsingController, EventRegistrationController, ClubMembershipController, and EventManagementController handle incoming requests and coordinate application workflows.

These controllers interact with service components, including AuthenticationService, DatabaseService, and NotificationService, which encapsulate core business logic and shared system functionality. The AuthenticationService manages user sessions and credential validation, the DatabaseService handles all data persistence operations, and the NotificationService manages system-generated updates and user notifications.

The Domain Model represents the core entities of the system, such as User, Club, Event, Announcement, and Registration, and serves as the foundation for data management. Additionally, the Filtering Module implements the Strategy and Factory design patterns to provide flexible and extensible event filtering capabilities. External dependencies, including the SQLite database and authentication system, are also shown, demonstrating how the backend

interacts with persistent storage and external identity services. Overall, this diagram highlights the modular structure of the system and the relationships between its key components.

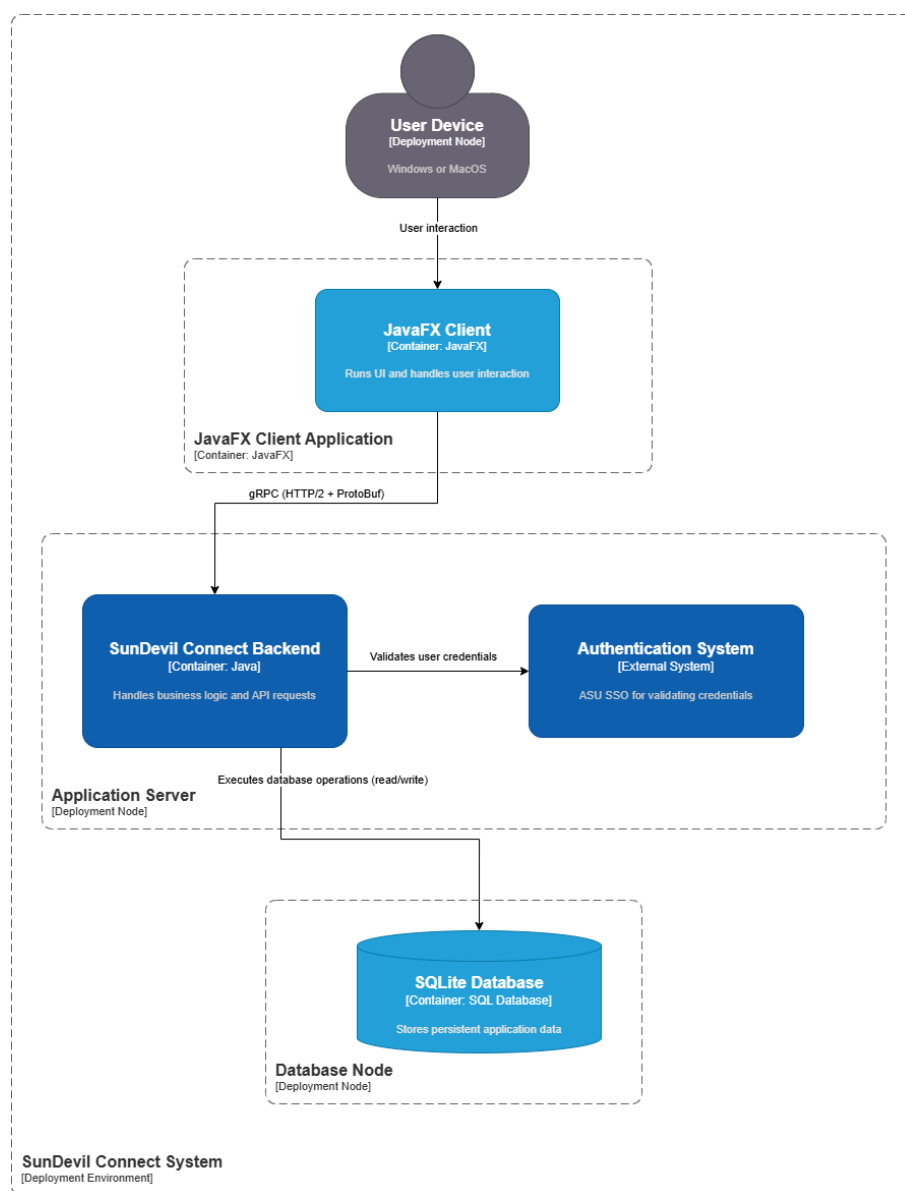


Deployment Diagram

The deployment diagram illustrates the physical distribution of the SunDevil Connect system across different runtime environments. The system is deployed across three primary nodes: the user device, the application server, and the database node. On the user device, the JavaFX client application runs locally and provides the graphical user interface through which users interact with the system.

The application server hosts the SunDevil Connect backend, which is implemented as a Java application responsible for processing requests, executing business logic, and coordinating system operations. Communication between the client and backend occurs via Protocol Buffers over TCP/HTTP, enabling efficient and structured data exchange.

The backend server interacts with the SQLite database, which resides within a dedicated database node and is responsible for storing persistent application data. Additionally, the system integrates with an external authentication system, such as ASU Single Sign-On, to validate user credentials and ensure secure access. This diagram clearly demonstrates how system components are deployed across different environments and how they communicate to support the overall functionality of the platform.



Video Link [5]: Video explaining implementation and dome phase I implementation.

<https://asu.zoom.us/rec/share/pANQQ6W-joKOhv-5CPgJZU-QOS8mHcC-pPjuRkMSEKnXsbDHN-uHWHX3SQfKuiog.k4JGMhbn6KHgkUN?startTime=1774985100000>

Passcode: DvD&66Y\$